Task 1: Sparse Networks from Scratch

In the first task, students will train sparse networks with a fixed random connectivity mask. The mask M is generated at initialization and remains unchanged throughout training. Students should compare at least two optimization methods: SGD with momentum and Adam. The goal is to investigate how different optimizers behave when training highly sparse networks. Suggested questions include:

- How does sparsity a!ect convergence speed?
- How does accuracy degrade as sparsity increases?
- Do different optimizers behave di!erently under high sparsity?

```
# from google.colab import drive
# drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/
```

Load libraries

```
import os
import sys
import torch
import torch.nn as nn
import torch.optim as optim
import joblib
import torchvision
import torchvision.transforms as transforms

import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
```

```
# quarto preview 01_task_1.ipynb --to pdf
```

```
# model_path = "/content/drive/MyDrive/Posgrados/2025/Duke/ECE 685D/Final project"
# sys.path.append(model_path)

model_path = "."

from models import ResNetCIFAR
```

```
use_pin_memory = torch.cuda.is_available()

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

num_workers = min(2, os.cpu_count())
```

Load data

```
# Standard CIFAR-10 transforms
transform = transforms.Compose(
    [
        transforms.ToTensor(),
        transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2023, 0.1994, 0.2010)),
    ]
)

trainset = torchvision.datasets.CIFAR10(
    root=f"{model_path}/data", train=True, download=True, transform=transform
)
trainloader = torch.utils.data.DataLoader(
    trainset,
    batch_size=128,
    shuffle=True,
    num_workers=num_workers,
    pin_memory=use_pin_memory,
)

testset = torchvision.datasets.CIFAR10(
    root=f"{model_path}/data", train=False, download=True, transform=transform
)
```

```
testloader = torch.utils.data.DataLoader(
    testset,
    batch_size=128,
    shuffle=False,
    num_workers=num_workers,
    pin_memory=use_pin_memory,
)
```

Configuration

```
sparsities = [0.90, 0.95, 0.98]
optimizers = ["sgd", "adam"]
```

Main Experiment

```
epochs = 15
criterion = nn.CrossEntropyLoss()
```

```
output_dir = "/content/drive/MyDrive/Posgrados/2025/Duke/ECE 685D/Final project/results"
```

Helper Functions

```
def get_static_masks(model, sparsity):
    """Generates the fixed binary mask M"""
    masks = {}
    for name, param in model.named_parameters():
        if "conv" in name or "linear" in name:
            mask = torch.rand(param.size()).to(device)
            mask = (mask > sparsity).float()
            masks[name] = mask
        else:
            masks[name] = torch.ones(param.size()).to(device)
    return masks


@torch.no_grad()
```

```
def apply_masks(model, masks):
    """Applies W = M * theta"""
    for name, param in model.named_parameters():
        param.mul_(masks[name])
```

Actual experiment

This section runs the full experimental loop over sparsity levels and optimizers. For each configuration, a model is initialized, a fixed random mask is applied, and training proceeds while enforcing the same sparse connectivity throughout. Performance metrics are tracked at every epoch and saved for later analysis.

```
for s in sparsities:
    for opt_name in optimizers:
        print(f"\nStarting Experiment: Sparsity={s*100}%, Optimizer={opt_name}")

        # Initialize
        model = ResNetCIFAR().to(device)
        masks = get_static_masks(model, s)
        apply_masks(model, masks)

        if opt_name == "sgd":
            optimizer = optim.SGD(
                model.parameters(), lr=0.1, momentum=0.9, weight_decay=5e-4
            )
        else:
            optimizer = optim.Adam(model.parameters(), lr=0.001, weight_decay=5e-4)

        history = {"train_loss": [], "train_acc": [], "test_loss": [], "test_acc": []}

        for epoch in range(epochs):
            # --- Training Phase ---
            model.train()
            train_loss, train_correct, train_total = 0.0, 0, 0

            for inputs, labels in trainloader:
                inputs, labels = inputs.to(device), labels.to(device)
                optimizer.zero_grad()
                outputs = model(inputs)
                loss = criterion(outputs, labels)
                loss.backward()
```

```
            optimizer.step()
            apply_masks(model, masks)  # Keep connectivity fixed

            train_loss += loss.item()
            _, predicted = outputs.max(1)
            train_total += labels.size(0)
            train_correct += predicted.eq(labels).sum().item()

        # --- Evaluation Phase ---
        model.eval()
        test_loss, test_correct, test_total = 0.0, 0, 0
        with torch.no_grad():
            for inputs, labels in testloader:
                inputs, labels = inputs.to(device), labels.to(device)
                outputs = model(inputs)
                loss = criterion(outputs, labels)
                test_loss += loss.item()
                _, predicted = outputs.max(1)
                test_total += labels.size(0)
                test_correct += predicted.eq(labels).sum().item()

        # Record Metrics
        history["train_loss"].append(train_loss / len(trainloader))
        history["train_acc"].append(100.0 * train_correct / train_total)
        history["test_loss"].append(test_loss / len(testloader))
        history["test_acc"].append(100.0 * test_correct / test_total)

        print(
            f"Epoch {epoch+1}: Train Loss {history['train_loss'][-1]:.4f} | Test Acc {his
        )

    # --- Save Joblib ---
    experiment_results = {
        "sparsity": s,
        "optimizer": opt_name,
        "train_loss": history["train_loss"],
        "train_acc": history["train_acc"],
        "test_loss": history["test_loss"],
        "test_acc": history["test_acc"],
        "masks": masks,  # Required deliverable [cite: 69]
        "model_state": model.state_dict(),  # For future analysis [cite: 67]
    }
```

```
        filename = os.path.join(
            output_dir, f"task_1_{opt_name}_{int(s*100)}_sparsity.joblib"
        )
        joblib.dump(experiment_results, filename)
        print(f"Results saved to {filename}!")
```

```
Starting Experiment: Sparsity=90.0%, Optimizer=sgd
Epoch 1: Train Loss 1.4800 | Test Acc 46.15%
Epoch 2: Train Loss 1.0508 | Test Acc 57.55%
Epoch 3: Train Loss 0.8884 | Test Acc 54.54%
Epoch 4: Train Loss 0.8074 | Test Acc 51.62%
Epoch 5: Train Loss 0.7596 | Test Acc 69.40%
Epoch 6: Train Loss 0.7299 | Test Acc 64.62%
Epoch 7: Train Loss 0.7096 | Test Acc 67.02%
Epoch 8: Train Loss 0.6902 | Test Acc 68.87%
Epoch 9: Train Loss 0.6775 | Test Acc 70.44%
Epoch 10: Train Loss 0.6648 | Test Acc 69.63%
Epoch 11: Train Loss 0.6513 | Test Acc 70.86%
Epoch 12: Train Loss 0.6478 | Test Acc 60.67%
Epoch 13: Train Loss 0.6390 | Test Acc 70.60%
Epoch 14: Train Loss 0.6302 | Test Acc 71.90%
Epoch 15: Train Loss 0.6326 | Test Acc 70.14%
Results saved to /content/drive/MyDrive/Posgrados/2025/Duke/ECE 685D/Final project/results/ta

Starting Experiment: Sparsity=90.0%, Optimizer=adam
Epoch 1: Train Loss 1.5871 | Test Acc 48.50%
Epoch 2: Train Loss 1.1645 | Test Acc 58.19%
Epoch 3: Train Loss 1.0149 | Test Acc 60.58%
Epoch 4: Train Loss 0.9248 | Test Acc 66.05%
Epoch 5: Train Loss 0.8605 | Test Acc 64.21%
Epoch 6: Train Loss 0.8024 | Test Acc 67.34%
Epoch 7: Train Loss 0.7640 | Test Acc 68.56%
Epoch 8: Train Loss 0.7243 | Test Acc 70.47%
Epoch 9: Train Loss 0.6920 | Test Acc 70.43%
Epoch 10: Train Loss 0.6628 | Test Acc 70.66%
Epoch 11: Train Loss 0.6403 | Test Acc 71.42%
Epoch 12: Train Loss 0.6167 | Test Acc 71.68%
Epoch 13: Train Loss 0.5946 | Test Acc 73.01%
Epoch 14: Train Loss 0.5739 | Test Acc 72.12%
Epoch 15: Train Loss 0.5620 | Test Acc 73.34%
```

```
Results saved to /content/drive/MyDrive/Posgrados/2025/Duke/ECE 685D/Final project/results/ta

Starting Experiment: Sparsity=95.0%, Optimizer=sgd
Epoch 1: Train Loss 1.6498 | Test Acc 38.35%
Epoch 2: Train Loss 1.2002 | Test Acc 57.12%
Epoch 3: Train Loss 1.0380 | Test Acc 59.63%
Epoch 4: Train Loss 0.9561 | Test Acc 60.00%
Epoch 5: Train Loss 0.9054 | Test Acc 52.98%
Epoch 6: Train Loss 0.8657 | Test Acc 65.12%
Epoch 7: Train Loss 0.8465 | Test Acc 62.64%
Epoch 8: Train Loss 0.8222 | Test Acc 63.60%
Epoch 9: Train Loss 0.8056 | Test Acc 65.55%
Epoch 10: Train Loss 0.7962 | Test Acc 67.65%
Epoch 11: Train Loss 0.7894 | Test Acc 66.20%
Epoch 12: Train Loss 0.7805 | Test Acc 55.36%
Epoch 13: Train Loss 0.7666 | Test Acc 60.81%
Epoch 14: Train Loss 0.7665 | Test Acc 67.60%
Epoch 15: Train Loss 0.7544 | Test Acc 61.46%
Results saved to /content/drive/MyDrive/Posgrados/2025/Duke/ECE 685D/Final project/results/ta

Starting Experiment: Sparsity=95.0%, Optimizer=adam
Epoch 1: Train Loss 1.8323 | Test Acc 36.19%
Epoch 2: Train Loss 1.5417 | Test Acc 41.56%
Epoch 3: Train Loss 1.4106 | Test Acc 42.55%
Epoch 4: Train Loss 1.3250 | Test Acc 45.08%
Epoch 5: Train Loss 1.2622 | Test Acc 48.91%
Epoch 6: Train Loss 1.2182 | Test Acc 49.05%
Epoch 7: Train Loss 1.1826 | Test Acc 50.83%
Epoch 8: Train Loss 1.1545 | Test Acc 54.83%
Epoch 9: Train Loss 1.1285 | Test Acc 51.85%
Epoch 10: Train Loss 1.1068 | Test Acc 56.14%
Epoch 11: Train Loss 1.0861 | Test Acc 56.15%
Epoch 12: Train Loss 1.0698 | Test Acc 57.18%
Epoch 13: Train Loss 1.0574 | Test Acc 55.74%
Epoch 14: Train Loss 1.0411 | Test Acc 57.95%
Epoch 15: Train Loss 1.0289 | Test Acc 57.92%
Results saved to /content/drive/MyDrive/Posgrados/2025/Duke/ECE 685D/Final project/results/ta

Starting Experiment: Sparsity=98.0%, Optimizer=sgd
Epoch 1: Train Loss 1.7946 | Test Acc 36.59%
Epoch 2: Train Loss 1.5153 | Test Acc 46.35%
Epoch 3: Train Loss 1.4071 | Test Acc 45.13%
Epoch 4: Train Loss 1.3515 | Test Acc 42.58%
```

```
Epoch 5: Train Loss 1.3171 | Test Acc 45.01%
Epoch 6: Train Loss 1.2975 | Test Acc 42.05%
Epoch 7: Train Loss 1.2746 | Test Acc 47.86%
Epoch 8: Train Loss 1.2583 | Test Acc 50.42%
Epoch 9: Train Loss 1.2576 | Test Acc 44.14%
Epoch 10: Train Loss 1.2435 | Test Acc 50.08%
Epoch 11: Train Loss 1.2302 | Test Acc 46.97%
Epoch 12: Train Loss 1.2275 | Test Acc 46.45%
Epoch 13: Train Loss 1.2237 | Test Acc 44.64%
Epoch 14: Train Loss 1.2136 | Test Acc 36.23%
Epoch 15: Train Loss 1.2156 | Test Acc 50.50%
Results saved to /content/drive/MyDrive/Posgrados/2025/Duke/ECE 685D/Final project/results/ta

Starting Experiment: Sparsity=98.0%, Optimizer=adam
Epoch 1: Train Loss 1.9828 | Test Acc 30.40%
Epoch 2: Train Loss 1.7337 | Test Acc 37.47%
Epoch 3: Train Loss 1.6010 | Test Acc 40.73%
Epoch 4: Train Loss 1.5292 | Test Acc 41.95%
Epoch 5: Train Loss 1.4710 | Test Acc 43.88%
Epoch 6: Train Loss 1.4234 | Test Acc 45.62%
Epoch 7: Train Loss 1.3807 | Test Acc 45.25%
Epoch 8: Train Loss 1.3495 | Test Acc 45.60%
Epoch 9: Train Loss 1.3253 | Test Acc 46.64%
Epoch 10: Train Loss 1.3034 | Test Acc 47.59%
Epoch 11: Train Loss 1.2862 | Test Acc 48.19%
Epoch 12: Train Loss 1.2712 | Test Acc 49.35%
Epoch 13: Train Loss 1.2562 | Test Acc 48.64%
Epoch 14: Train Loss 1.2407 | Test Acc 46.10%
Epoch 15: Train Loss 1.2298 | Test Acc 47.28%
Results saved to /content/drive/MyDrive/Posgrados/2025/Duke/ECE 685D/Final project/results/ta
```

The printed outputs provide a quick view of training loss and test accuracy across epochs for each configuration. As sparsity increases, performance generally degrades, reflecting the reduced capacity of the network. Differences between optimizers also become more pronounced at higher sparsities, with Adam often converging more smoothly than SGD.

These logs only show a subset of the recorded metrics. Additional information such as training accuracy, test loss, and the full training history are stored in the saved joblib files for more detailed analysis.

Evaluation

Functions

```
def plot_training_and_validation(results):
    sns.set_theme(style="whitegrid", context="notebook", palette="deep")

    sparsities = [0.90, 0.95, 0.98]
    opts = ["sgd", "adam"]

    for s in sparsities:
        rows = []

        # --- Reshape data into tidy format ---
        for opt in opts:
            if s in results[opt]:
                n_epochs = len(results[opt][s]["train_loss"])
                for epoch in range(n_epochs):
                    rows.append(
                        {
                            "epoch": epoch,
                            "optimizer": opt.upper(),
                            "split": "Train",
                            "loss": results[opt][s]["train_loss"][epoch],
                            "accuracy": results[opt][s]["train_acc"][epoch],
                        }
                    )
                    rows.append(
                        {
                            "epoch": epoch,
                            "optimizer": opt.upper(),
                            "split": "Test",
                            "loss": results[opt][s]["test_loss"][epoch],
                            "accuracy": results[opt][s]["test_acc"][epoch],
                        }
                    )

        df = pd.DataFrame(rows)

        # --- Create 2x2 figure ---
        fig, axes = plt.subplots(2, 2, figsize=(14, 10), sharex=True)
```

```
        fig.suptitle(
            f"Optimization Analysis at {int(s*100)}% Sparsity",
            fontsize=18,
            weight="bold",
        )

        # --- Train Loss ---
        sns.lineplot(
            data=df[df["split"] == "Train"],
            x="epoch",
            y="loss",
            hue="optimizer",
            linewidth=2.5,
            ax=axes[0, 0],
        )
        axes[0, 0].set_title("Train Loss")
        axes[0, 0].set_ylabel("Loss")
        axes[0, 0].legend(title="")

        # --- Test Loss ---
        sns.lineplot(
            data=df[df["split"] == "Test"],
            x="epoch",
            y="loss",
            hue="optimizer",
            linewidth=2.5,
            ax=axes[0, 1],
        )
        axes[0, 1].set_title("Test Loss")
        axes[0, 1].set_ylabel("Loss")
        axes[0, 1].legend(title="")

        # --- Train Accuracy ---
        sns.lineplot(
            data=df[df["split"] == "Train"],
            x="epoch",
            y="accuracy",
            hue="optimizer",
            linewidth=2.5,
            ax=axes[1, 0],
        )
        axes[1, 0].set_title("Train Accuracy")
```

```
        axes[1, 0].set_xlabel("Epoch")
        axes[1, 0].set_ylabel("Accuracy (%)")
        axes[1, 0].legend(title="")

        # --- Test Accuracy ---
        sns.lineplot(
            data=df[df["split"] == "Test"],
            x="epoch",
            y="accuracy",
            hue="optimizer",
            linewidth=2.5,
            ax=axes[1, 1],
        )
        axes[1, 1].set_title(
            "Test Accuracy",
        )
        axes[1, 1].set_xlabel("Epoch")
        axes[1, 1].set_ylabel("Accuracy (%)")
        axes[1, 1].legend(title="")

        # --- Final polish ---
        for ax in axes.flat:
            ax.grid(True, alpha=0.3)

        sns.despine()
        plt.tight_layout()

        plt.savefig(
            f"img/task_1_sparsity_{int(s*100)}.png", dpi=300, bbox_inches="tight"
        )
        plt.show()
```

```
def plot_accuracy_degradation(results):
    sns.set_theme(style="whitegrid", context="paper", palette="deep")

    sparsities = [0.90, 0.95, 0.98]
    opts = ["sgd", "adam"]

    # --- Build tidy dataframe ---
    rows = []
    for opt in opts:
        for s in sparsities:
```

```
            if s in results[opt]:
                rows.append(
                    {
                        "optimizer": opt.upper(),
                        "sparsity": int(s * 100),
                        "final_acc": results[opt][s]["test_acc"][-1],
                    }
                )

    df = pd.DataFrame(rows)

    fig, ax = plt.subplots(figsize=(5, 3))

    sns.lineplot(
        data=df,
        x="sparsity",
        y="final_acc",
        hue="optimizer",
        marker="o",
        linewidth=2.5,
        ax=ax,
    )

    ax.set_title(
        "Impact of Sparsity on Final Test Accuracy",
        # fontsize=16,
        weight="bold",
    )
    ax.set_xlabel("Sparsity Level (%)")
    ax.set_ylabel("Final Test Accuracy (%)")

    ax.set_xticks([90, 95, 98])
    ax.legend(title="")

    ax.grid(True, alpha=0.3)
    sns.despine()

    plt.tight_layout()
    plt.savefig("img/task_1_accuracy_vs_sparsity.png", dpi=300, bbox_inches="tight")
    plt.show()
```

```python
def report_layer_sparsity(results, opt="sgd", s=0.98):
    """
    Quantifies the sparsity level of different network components.
    Formula: sparsity(W) = 1 - (||W||_0 / |W|)
    """
    if opt in results and s in results[opt]:
        masks = results[opt][s]["masks"]

        print(f"--- Sparsity Statistics: {opt.upper()} | {int(s*100)}% Target ---")
        print(f"{'Layer Name':<35} | {'Active Weights':<15} | {'Actual Sparsity':<15}")
        print("-" * 70)

        total_params = 0
        total_nonzero = 0

        for name, mask in masks.items():
            # Calculate metrics using the project formula [cite: 55, 57]
            n_params = mask.numel()
            n_nonzero = torch.count_nonzero(mask).item()

            # Project formula: 1 - (nonzero / total)
            layer_sparsity = 1.0 - (n_nonzero / n_params)

            # Accumulate for global stats
            total_params += n_params
            total_nonzero += n_nonzero

            print(f"{name:<35} | {n_nonzero:<15} | {layer_sparsity:.4f}")

        overall_sparsity = 1.0 - (total_nonzero / total_params)
        print("-" * 70)
        print(
            f"{'TOTAL NETWORK SPARSITY':<35} | {total_nonzero:<15} | {overall_sparsity:.4f}\n
        )
    else:
        print(f"Data for {opt} at {s} sparsity not found in results.")
```

```python
def visualize_kernels(results, opt="sgd", s=0.90, layer_name="layer1.0.conv1.weight"):
    sns.set_theme(style="white", context="notebook")

    if opt in results and s in results[opt]:
```

```
        weights = results[opt][s]["model_state"][layer_name]

        kernels = weights[:16, 0, :, :].detach().cpu().numpy()

        fig, axes = plt.subplots(4, 4, figsize=(6, 6))

        fig.suptitle(
            f"Kernels ({layer_name})\n{opt.upper()} | {int(s*100)}% Sparsity",
            fontsize=14,
            weight="bold",
        )

        ims = []

        for i, ax in enumerate(axes.flat):
            im = ax.imshow(kernels[i], cmap="viridis", interpolation="nearest")
            ims.append(im)

            ax.set_title(f"F{i+1}", fontsize=8)
            ax.axis("off")

        # --- shared colorbar (makes comparison meaningful) ---
        cbar = fig.colorbar(ims[0], ax=axes, fraction=0.025, pad=0.02)
        cbar.ax.tick_params(labelsize=8)

        plt.tight_layout()
        plt.show()

    else:
        print(f"Data for {opt} at {s} sparsity not found.")
```

Read results

```
output_dir = "results"
```

```
# Patch torch.load to force CPU
_original_torch_load = torch.load


def cpu_torch_load(*args, **kwargs):
```

```
    kwargs["map_location"] = torch.device("cpu")
    return _original_torch_load(*args, **kwargs)


torch.load = cpu_torch_load

results = {}
```

```
for opt_name in optimizers:
    results[opt_name] = {}
    for s in sparsities:
        file_path = os.path.join(
            output_dir, f"task_1_{opt_name}_{int(s*100)}_sparsity.joblib"
        )

        if os.path.exists(file_path):
            data = joblib.load(file_path)
            results[opt_name][s] = data
            print(f"Loaded: {file_path}")
        else:
            print(f"Missing: {file_path}")
```

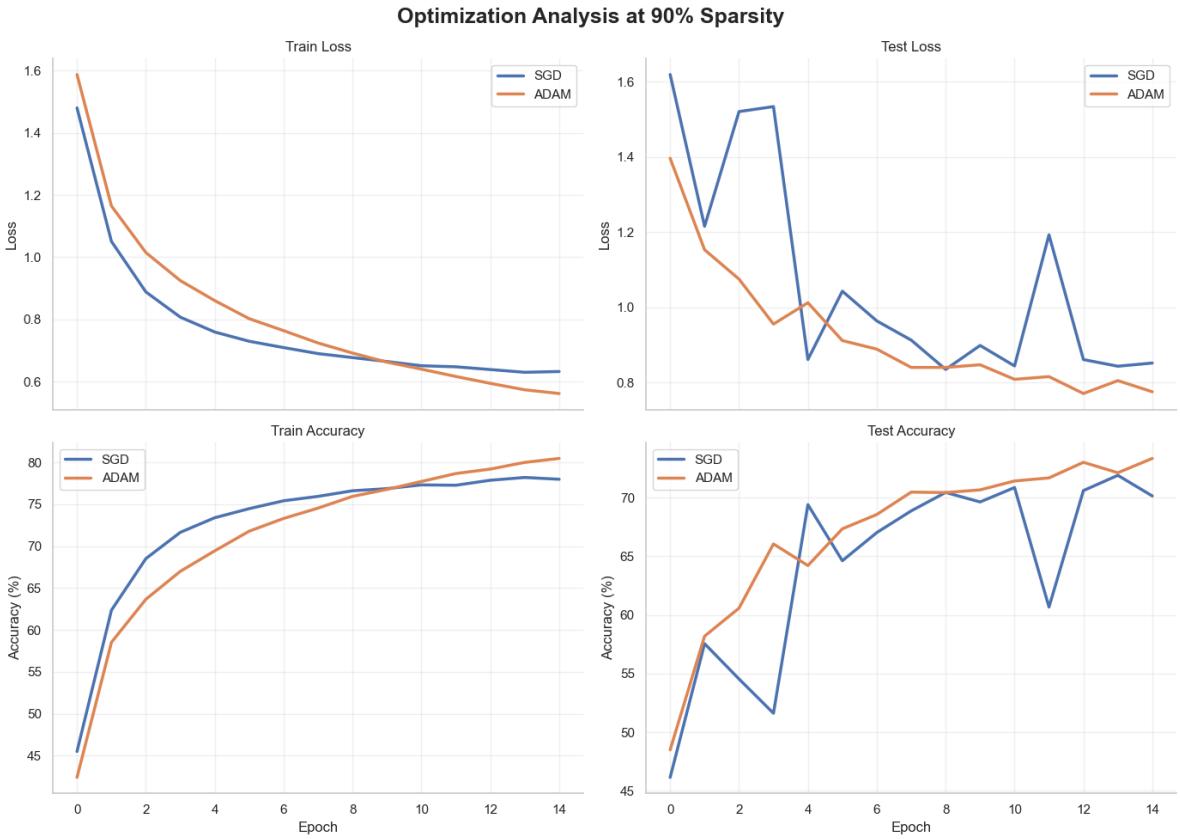
```
Loaded: results/task_1_sgd_90_sparsity.joblib
Loaded: results/task_1_sgd_95_sparsity.joblib
Loaded: results/task_1_sgd_98_sparsity.joblib
Loaded: results/task_1_adam_90_sparsity.joblib
Loaded: results/task_1_adam_95_sparsity.joblib
Loaded: results/task_1_adam_98_sparsity.joblib
```

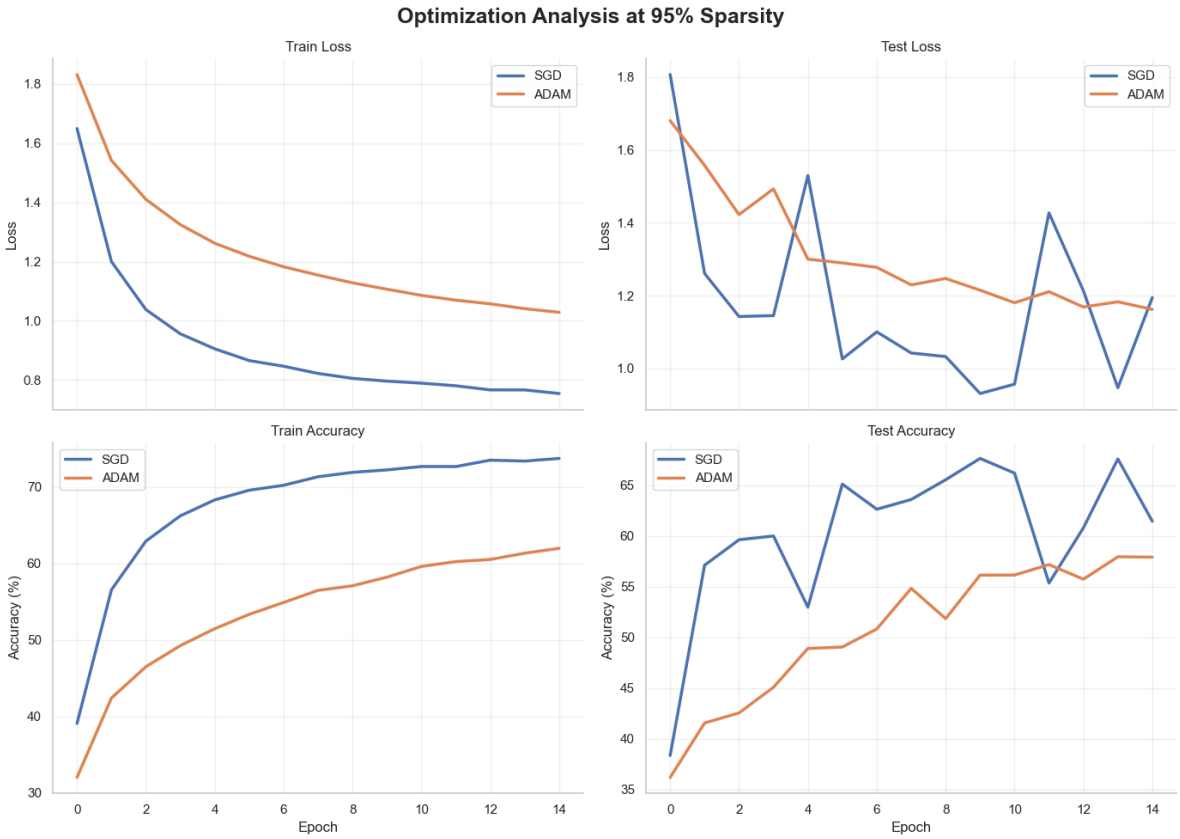
Analysis

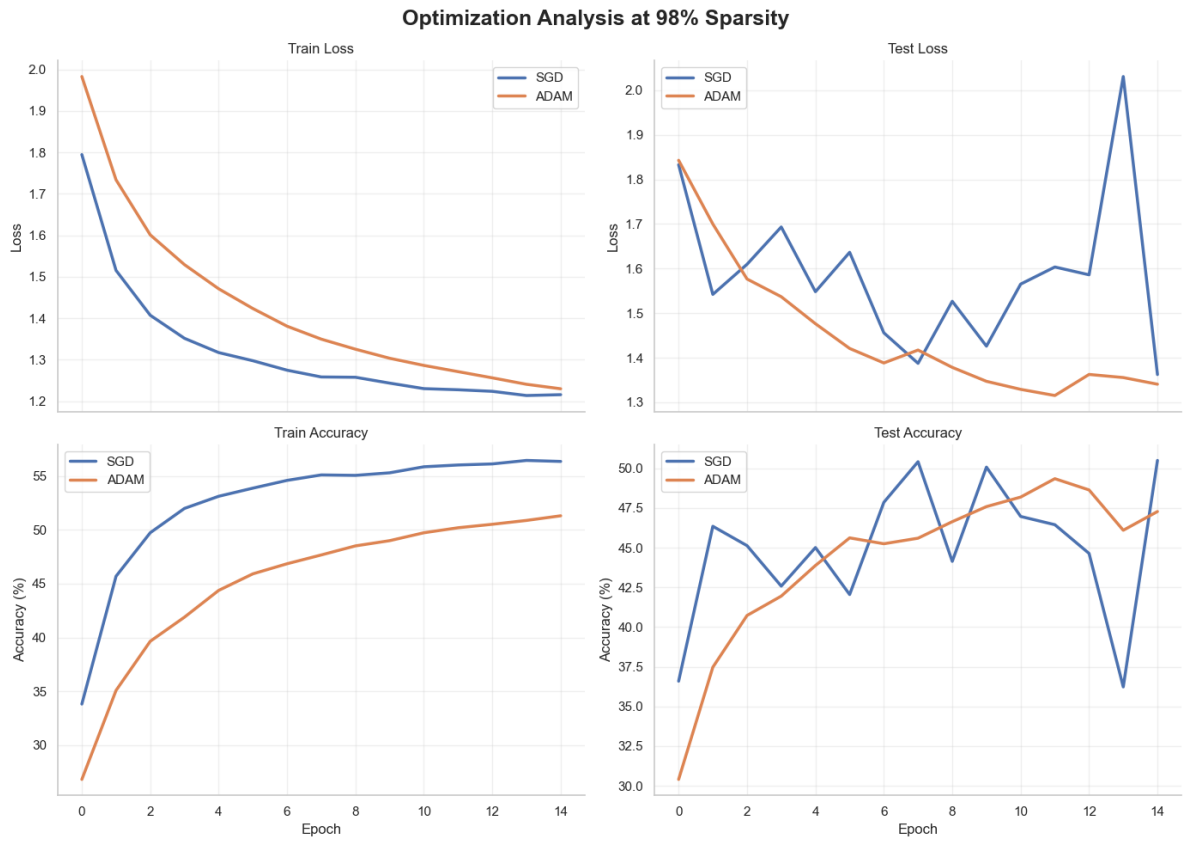
Training and Validation Curves

- How does sparsity affect convergence speed?

```
plot_training_and_validation(results)
```


Optimization Analysis at 90% Sparsity
Train Loss
Test Loss
Train Accuracy
Test Accuracy
SGD
ADAM
Loss
Accuracy (%)
Epoch

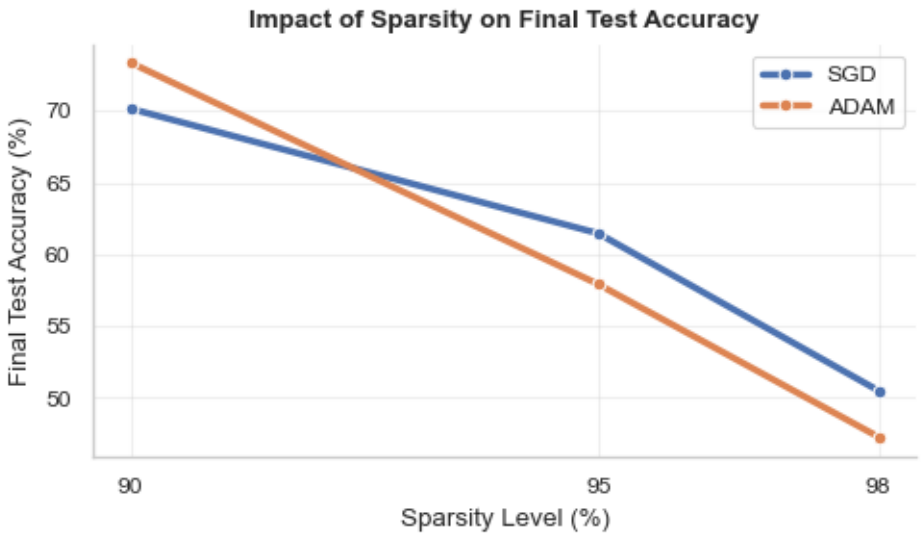

Optimization Analysis at 95% Sparsity
Train Loss
Test Loss
Train Accuracy
Test Accuracy
SGD
ADAM
Loss
Accuracy (%)
Epoch



Accuracy vs. Sparsity (Degradation)

- How does accuracy degrade as sparsity increases?

```
plot_accuracy_degradation(results)
```



Layer sparsity

This is from the evaluation metrics

```
for opt in optimizers:
    for s in sparsities:
        report_layer_sparsity(results, opt=opt, s=s)
```

```
--- Sparsity Statistics: SGD | 90% Target ---
Layer Name                          | Active Weights  | Actual Sparsity
-----------------------------------------------------------------------
conv1.weight                        | 53              | 0.8773
bn1.weight                          | 16              | 0.0000
bn1.bias                            | 16              | 0.0000
layer1.0.conv1.weight               | 229             | 0.9006
layer1.0.bn1.weight                 | 16              | 0.0000
layer1.0.bn1.bias                   | 16              | 0.0000
layer1.0.conv2.weight               | 225             | 0.9023
layer1.0.bn2.weight                 | 16              | 0.0000
layer1.0.bn2.bias                   | 16              | 0.0000
layer1.1.conv1.weight               | 240             | 0.8958
layer1.1.bn1.weight                 | 16              | 0.0000
layer1.1.bn1.bias                   | 16              | 0.0000
layer1.1.conv2.weight               | 242             | 0.8950
layer1.1.bn2.weight                 | 16              | 0.0000
```

layer1.1.bn2.bias	16	0.0000
layer1.2.conv1.weight	219	0.9049
layer1.2.bn1.weight	16	0.0000
layer1.2.bn1.bias	16	0.0000
layer1.2.conv2.weight	257	0.8885
layer1.2.bn2.weight	16	0.0000
layer1.2.bn2.bias	16	0.0000
layer2.0.conv1.weight	502	0.8911
layer2.0.bn1.weight	32	0.0000
layer2.0.bn1.bias	32	0.0000
layer2.0.conv2.weight	948	0.8971
layer2.0.bn2.weight	32	0.0000
layer2.0.bn2.bias	32	0.0000
layer2.0.shortcut.0.weight	512	0.0000
layer2.0.shortcut.1.weight	32	0.0000
layer2.0.shortcut.1.bias	32	0.0000
layer2.1.conv1.weight	902	0.9021
layer2.1.bn1.weight	32	0.0000
layer2.1.bn1.bias	32	0.0000
layer2.1.conv2.weight	891	0.9033
layer2.1.bn2.weight	32	0.0000
layer2.1.bn2.bias	32	0.0000
layer2.2.conv1.weight	949	0.8970
layer2.2.bn1.weight	32	0.0000
layer2.2.bn1.bias	32	0.0000
layer2.2.conv2.weight	922	0.9000
layer2.2.bn2.weight	32	0.0000
layer2.2.bn2.bias	32	0.0000
layer3.0.conv1.weight	1849	0.8997
layer3.0.bn1.weight	64	0.0000
layer3.0.bn1.bias	64	0.0000
layer3.0.conv2.weight	3653	0.9009
layer3.0.bn2.weight	64	0.0000
layer3.0.bn2.bias	64	0.0000
layer3.0.shortcut.0.weight	2048	0.0000
layer3.0.shortcut.1.weight	64	0.0000
layer3.0.shortcut.1.bias	64	0.0000
layer3.1.conv1.weight	3745	0.8984
layer3.1.bn1.weight	64	0.0000
layer3.1.bn1.bias	64	0.0000
layer3.1.conv2.weight	3634	0.9014
layer3.1.bn2.weight	64	0.0000
layer3.1.bn2.bias	64	0.0000

```
layer3.2.conv1.weight               | 3686           | 0.9000
layer3.2.bn1.weight                 | 64             | 0.0000
layer3.2.bn1.bias                   | 64             | 0.0000
layer3.2.conv2.weight               | 3660           | 0.9007
layer3.2.bn2.weight                 | 64             | 0.0000
layer3.2.bn2.bias                   | 64             | 0.0000
linear.weight                       | 61             | 0.9047
linear.bias                         | 0              | 1.0000
-----------------------------------------------------------------------
TOTAL NETWORK SPARSITY              | 30995          | 0.8862


--- Sparsity Statistics: SGD | 95% Target ---
Layer Name                          | Active Weights  | Actual Sparsity
-----------------------------------------------------------------------
conv1.weight                        | 24             | 0.9444
bn1.weight                          | 16             | 0.0000
bn1.bias                            | 16             | 0.0000
layer1.0.conv1.weight               | 132            | 0.9427
layer1.0.bn1.weight                 | 16             | 0.0000
layer1.0.bn1.bias                   | 16             | 0.0000
layer1.0.conv2.weight               | 121            | 0.9475
layer1.0.bn2.weight                 | 16             | 0.0000
layer1.0.bn2.bias                   | 16             | 0.0000
layer1.1.conv1.weight               | 117            | 0.9492
layer1.1.bn1.weight                 | 16             | 0.0000
layer1.1.bn1.bias                   | 16             | 0.0000
layer1.1.conv2.weight               | 136            | 0.9410
layer1.1.bn2.weight                 | 16             | 0.0000
layer1.1.bn2.bias                   | 16             | 0.0000
layer1.2.conv1.weight               | 129            | 0.9440
layer1.2.bn1.weight                 | 16             | 0.0000
layer1.2.bn1.bias                   | 16             | 0.0000
layer1.2.conv2.weight               | 123            | 0.9466
layer1.2.bn2.weight                 | 16             | 0.0000
layer1.2.bn2.bias                   | 16             | 0.0000
layer2.0.conv1.weight               | 246            | 0.9466
layer2.0.bn1.weight                 | 32             | 0.0000
layer2.0.bn1.bias                   | 32             | 0.0000
layer2.0.conv2.weight               | 494            | 0.9464
layer2.0.bn2.weight                 | 32             | 0.0000
layer2.0.bn2.bias                   | 32             | 0.0000
layer2.0.shortcut.0.weight          | 512            | 0.0000
```

```
layer2.0.shortcut.1.weight          | 32              | 0.0000
layer2.0.shortcut.1.bias            | 32              | 0.0000
layer2.1.conv1.weight               | 455             | 0.9506
layer2.1.bn1.weight                 | 32              | 0.0000
layer2.1.bn1.bias                   | 32              | 0.0000
layer2.1.conv2.weight               | 461             | 0.9500
layer2.1.bn2.weight                 | 32              | 0.0000
layer2.1.bn2.bias                   | 32              | 0.0000
layer2.2.conv1.weight               | 456             | 0.9505
layer2.2.bn1.weight                 | 32              | 0.0000
layer2.2.bn1.bias                   | 32              | 0.0000
layer2.2.conv2.weight               | 454             | 0.9507
layer2.2.bn2.weight                 | 32              | 0.0000
layer2.2.bn2.bias                   | 32              | 0.0000
layer3.0.conv1.weight               | 919             | 0.9501
layer3.0.bn1.weight                 | 64              | 0.0000
layer3.0.bn1.bias                   | 64              | 0.0000
layer3.0.conv2.weight               | 1834            | 0.9502
layer3.0.bn2.weight                 | 64              | 0.0000
layer3.0.bn2.bias                   | 64              | 0.0000
layer3.0.shortcut.0.weight          | 2048            | 0.0000
layer3.0.shortcut.1.weight          | 64              | 0.0000
layer3.0.shortcut.1.bias            | 64              | 0.0000
layer3.1.conv1.weight               | 1850            | 0.9498
layer3.1.bn1.weight                 | 64              | 0.0000
layer3.1.bn1.bias                   | 64              | 0.0000
layer3.1.conv2.weight               | 1850            | 0.9498
layer3.1.bn2.weight                 | 64              | 0.0000
layer3.1.bn2.bias                   | 64              | 0.0000
layer3.2.conv1.weight               | 1870            | 0.9493
layer3.2.bn1.weight                 | 64              | 0.0000
layer3.2.bn1.bias                   | 64              | 0.0000
layer3.2.conv2.weight               | 1879            | 0.9490
layer3.2.bn2.weight                 | 64              | 0.0000
layer3.2.bn2.bias                   | 64              | 0.0000
linear.weight                       | 28              | 0.9563
linear.bias                         | 0               | 1.0000
--------------------------------------------------------------------
TOTAL NETWORK SPARSITY              | 17706           | 0.9350


--- Sparsity Statistics: SGD | 98% Target ---
Layer Name                          | Active Weights  | Actual Sparsity
```

conv1.weight	11	0.9745
bn1.weight	16	0.0000
bn1.bias	16	0.0000
layer1.0.conv1.weight	28	0.9878
layer1.0.bn1.weight	16	0.0000
layer1.0.bn1.bias	16	0.0000
layer1.0.conv2.weight	44	0.9809
layer1.0.bn2.weight	16	0.0000
layer1.0.bn2.bias	16	0.0000
layer1.1.conv1.weight	39	0.9831
layer1.1.bn1.weight	16	0.0000
layer1.1.bn1.bias	16	0.0000
layer1.1.conv2.weight	49	0.9787
layer1.1.bn2.weight	16	0.0000
layer1.1.bn2.bias	16	0.0000
layer1.2.conv1.weight	47	0.9796
layer1.2.bn1.weight	16	0.0000
layer1.2.bn1.bias	16	0.0000
layer1.2.conv2.weight	51	0.9779
layer1.2.bn2.weight	16	0.0000
layer1.2.bn2.bias	16	0.0000
layer2.0.conv1.weight	98	0.9787
layer2.0.bn1.weight	32	0.0000
layer2.0.bn1.bias	32	0.0000
layer2.0.conv2.weight	183	0.9801
layer2.0.bn2.weight	32	0.0000
layer2.0.bn2.bias	32	0.0000
layer2.0.shortcut.0.weight	512	0.0000
layer2.0.shortcut.1.weight	32	0.0000
layer2.0.shortcut.1.bias	32	0.0000
layer2.1.conv1.weight	195	0.9788
layer2.1.bn1.weight	32	0.0000
layer2.1.bn1.bias	32	0.0000
layer2.1.conv2.weight	193	0.9791
layer2.1.bn2.weight	32	0.0000
layer2.1.bn2.bias	32	0.0000
layer2.2.conv1.weight	187	0.9797
layer2.2.bn1.weight	32	0.0000
layer2.2.bn1.bias	32	0.0000
layer2.2.conv2.weight	193	0.9791
layer2.2.bn2.weight	32	0.0000
layer2.2.bn2.bias	32	0.0000

```
layer3.0.conv1.weight               | 361             | 0.9804
layer3.0.bn1.weight                 | 64              | 0.0000
layer3.0.bn1.bias                   | 64              | 0.0000
layer3.0.conv2.weight               | 759             | 0.9794
layer3.0.bn2.weight                 | 64              | 0.0000
layer3.0.bn2.bias                   | 64              | 0.0000
layer3.0.shortcut.0.weight          | 2048            | 0.0000
layer3.0.shortcut.1.weight          | 64              | 0.0000
layer3.0.shortcut.1.bias            | 64              | 0.0000
layer3.1.conv1.weight               | 740             | 0.9799
layer3.1.bn1.weight                 | 64              | 0.0000
layer3.1.bn1.bias                   | 64              | 0.0000
layer3.1.conv2.weight               | 698             | 0.9811
layer3.1.bn2.weight                 | 64              | 0.0000
layer3.1.bn2.bias                   | 64              | 0.0000
layer3.2.conv1.weight               | 754             | 0.9795
layer3.2.bn1.weight                 | 64              | 0.0000
layer3.2.bn1.bias                   | 64              | 0.0000
layer3.2.conv2.weight               | 721             | 0.9804
layer3.2.bn2.weight                 | 64              | 0.0000
layer3.2.bn2.bias                   | 64              | 0.0000
linear.weight                       | 15              | 0.9766
linear.bias                         | 1               | 0.9000
-----------------------------------------------------------------------
TOTAL NETWORK SPARSITY              | 9495            | 0.9652


--- Sparsity Statistics: ADAM | 90% Target ---
Layer Name                          | Active Weights  | Actual Sparsity
-----------------------------------------------------------------------
conv1.weight                        | 48              | 0.8889
bn1.weight                          | 16              | 0.0000
bn1.bias                            | 16              | 0.0000
layer1.0.conv1.weight               | 245             | 0.8937
layer1.0.bn1.weight                 | 16              | 0.0000
layer1.0.bn1.bias                   | 16              | 0.0000
layer1.0.conv2.weight               | 230             | 0.9002
layer1.0.bn2.weight                 | 16              | 0.0000
layer1.0.bn2.bias                   | 16              | 0.0000
layer1.1.conv1.weight               | 235             | 0.8980
layer1.1.bn1.weight                 | 16              | 0.0000
layer1.1.bn1.bias                   | 16              | 0.0000
layer1.1.conv2.weight               | 248             | 0.8924
```

layer1.1.bn2.weight	16	0.0000
layer1.1.bn2.bias	16	0.0000
layer1.2.conv1.weight	220	0.9045
layer1.2.bn1.weight	16	0.0000
layer1.2.bn1.bias	16	0.0000
layer1.2.conv2.weight	201	0.9128
layer1.2.bn2.weight	16	0.0000
layer1.2.bn2.bias	16	0.0000
layer2.0.conv1.weight	471	0.8978
layer2.0.bn1.weight	32	0.0000
layer2.0.bn1.bias	32	0.0000
layer2.0.conv2.weight	935	0.8985
layer2.0.bn2.weight	32	0.0000
layer2.0.bn2.bias	32	0.0000
layer2.0.shortcut.0.weight	512	0.0000
layer2.0.shortcut.1.weight	32	0.0000
layer2.0.shortcut.1.bias	32	0.0000
layer2.1.conv1.weight	920	0.9002
layer2.1.bn1.weight	32	0.0000
layer2.1.bn1.bias	32	0.0000
layer2.1.conv2.weight	896	0.9028
layer2.1.bn2.weight	32	0.0000
layer2.1.bn2.bias	32	0.0000
layer2.2.conv1.weight	922	0.9000
layer2.2.bn1.weight	32	0.0000
layer2.2.bn1.bias	32	0.0000
layer2.2.conv2.weight	936	0.8984
layer2.2.bn2.weight	32	0.0000
layer2.2.bn2.bias	32	0.0000
layer3.0.conv1.weight	1869	0.8986
layer3.0.bn1.weight	64	0.0000
layer3.0.bn1.bias	64	0.0000
layer3.0.conv2.weight	3745	0.8984
layer3.0.bn2.weight	64	0.0000
layer3.0.bn2.bias	64	0.0000
layer3.0.shortcut.0.weight	2048	0.0000
layer3.0.shortcut.1.weight	64	0.0000
layer3.0.shortcut.1.bias	64	0.0000
layer3.1.conv1.weight	3678	0.9002
layer3.1.bn1.weight	64	0.0000
layer3.1.bn1.bias	64	0.0000
layer3.1.conv2.weight	3667	0.9005
layer3.1.bn2.weight	64	0.0000

```
layer3.1.bn2.bias                   | 64              | 0.0000
layer3.2.conv1.weight               | 3715            | 0.8992
layer3.2.bn1.weight                 | 64              | 0.0000
layer3.2.bn1.bias                   | 64              | 0.0000
layer3.2.conv2.weight               | 3771            | 0.8977
layer3.2.bn2.weight                 | 64              | 0.0000
layer3.2.bn2.bias                   | 64              | 0.0000
linear.weight                       | 66              | 0.8969
linear.bias                         | 3               | 0.7000
-----------------------------------------------------------------------
TOTAL NETWORK SPARSITY              | 31149           | 0.8857


--- Sparsity Statistics: ADAM | 95% Target ---
Layer Name                          | Active Weights  | Actual Sparsity
-----------------------------------------------------------------------
conv1.weight                        | 26              | 0.9398
bn1.weight                          | 16              | 0.0000
bn1.bias                            | 16              | 0.0000
layer1.0.conv1.weight               | 102             | 0.9557
layer1.0.bn1.weight                 | 16              | 0.0000
layer1.0.bn1.bias                   | 16              | 0.0000
layer1.0.conv2.weight               | 132             | 0.9427
layer1.0.bn2.weight                 | 16              | 0.0000
layer1.0.bn2.bias                   | 16              | 0.0000
layer1.1.conv1.weight               | 104             | 0.9549
layer1.1.bn1.weight                 | 16              | 0.0000
layer1.1.bn1.bias                   | 16              | 0.0000
layer1.1.conv2.weight               | 130             | 0.9436
layer1.1.bn2.weight                 | 16              | 0.0000
layer1.1.bn2.bias                   | 16              | 0.0000
layer1.2.conv1.weight               | 114             | 0.9505
layer1.2.bn1.weight                 | 16              | 0.0000
layer1.2.bn1.bias                   | 16              | 0.0000
layer1.2.conv2.weight               | 109             | 0.9527
layer1.2.bn2.weight                 | 16              | 0.0000
layer1.2.bn2.bias                   | 16              | 0.0000
layer2.0.conv1.weight               | 261             | 0.9434
layer2.0.bn1.weight                 | 32              | 0.0000
layer2.0.bn1.bias                   | 32              | 0.0000
layer2.0.conv2.weight               | 466             | 0.9494
layer2.0.bn2.weight                 | 32              | 0.0000
layer2.0.bn2.bias                   | 32              | 0.0000
```

```
layer2.0.shortcut.0.weight          | 512             | 0.0000
layer2.0.shortcut.1.weight          | 32              | 0.0000
layer2.0.shortcut.1.bias            | 32              | 0.0000
layer2.1.conv1.weight               | 450             | 0.9512
layer2.1.bn1.weight                 | 32              | 0.0000
layer2.1.bn1.bias                   | 32              | 0.0000
layer2.1.conv2.weight               | 518             | 0.9438
layer2.1.bn2.weight                 | 32              | 0.0000
layer2.1.bn2.bias                   | 32              | 0.0000
layer2.2.conv1.weight               | 426             | 0.9538
layer2.2.bn1.weight                 | 32              | 0.0000
layer2.2.bn1.bias                   | 32              | 0.0000
layer2.2.conv2.weight               | 477             | 0.9482
layer2.2.bn2.weight                 | 32              | 0.0000
layer2.2.bn2.bias                   | 32              | 0.0000
layer3.0.conv1.weight               | 876             | 0.9525
layer3.0.bn1.weight                 | 64              | 0.0000
layer3.0.bn1.bias                   | 64              | 0.0000
layer3.0.conv2.weight               | 1835            | 0.9502
layer3.0.bn2.weight                 | 64              | 0.0000
layer3.0.bn2.bias                   | 64              | 0.0000
layer3.0.shortcut.0.weight          | 2048            | 0.0000
layer3.0.shortcut.1.weight          | 64              | 0.0000
layer3.0.shortcut.1.bias            | 64              | 0.0000
layer3.1.conv1.weight               | 1844            | 0.9500
layer3.1.bn1.weight                 | 64              | 0.0000
layer3.1.bn1.bias                   | 64              | 0.0000
layer3.1.conv2.weight               | 1875            | 0.9491
layer3.1.bn2.weight                 | 64              | 0.0000
layer3.1.bn2.bias                   | 64              | 0.0000
layer3.2.conv1.weight               | 1804            | 0.9511
layer3.2.bn1.weight                 | 64              | 0.0000
layer3.2.bn1.bias                   | 64              | 0.0000
layer3.2.conv2.weight               | 1787            | 0.9515
layer3.2.bn2.weight                 | 64              | 0.0000
layer3.2.bn2.bias                   | 64              | 0.0000
linear.weight                       | 23              | 0.9641
linear.bias                         | 2               | 0.8000
----------------------------------------------------------------------
TOTAL NETWORK SPARSITY              | 17489           | 0.9358


--- Sparsity Statistics: ADAM | 98% Target ---
```

Layer Name	Active Weights	Actual Sparsity
conv1.weight	11	0.9745
bn1.weight	16	0.0000
bn1.bias	16	0.0000
layer1.0.conv1.weight	64	0.9722
layer1.0.bn1.weight	16	0.0000
layer1.0.bn1.bias	16	0.0000
layer1.0.conv2.weight	62	0.9731
layer1.0.bn2.weight	16	0.0000
layer1.0.bn2.bias	16	0.0000
layer1.1.conv1.weight	44	0.9809
layer1.1.bn1.weight	16	0.0000
layer1.1.bn1.bias	16	0.0000
layer1.1.conv2.weight	52	0.9774
layer1.1.bn2.weight	16	0.0000
layer1.1.bn2.bias	16	0.0000
layer1.2.conv1.weight	44	0.9809
layer1.2.bn1.weight	16	0.0000
layer1.2.bn1.bias	16	0.0000
layer1.2.conv2.weight	37	0.9839
layer1.2.bn2.weight	16	0.0000
layer1.2.bn2.bias	16	0.0000
layer2.0.conv1.weight	91	0.9803
layer2.0.bn1.weight	32	0.0000
layer2.0.bn1.bias	32	0.0000
layer2.0.conv2.weight	180	0.9805
layer2.0.bn2.weight	32	0.0000
layer2.0.bn2.bias	32	0.0000
layer2.0.shortcut.0.weight	512	0.0000
layer2.0.shortcut.1.weight	32	0.0000
layer2.0.shortcut.1.bias	32	0.0000
layer2.1.conv1.weight	176	0.9809
layer2.1.bn1.weight	32	0.0000
layer2.1.bn1.bias	32	0.0000
layer2.1.conv2.weight	181	0.9804
layer2.1.bn2.weight	32	0.0000
layer2.1.bn2.bias	32	0.0000
layer2.2.conv1.weight	195	0.9788
layer2.2.bn1.weight	32	0.0000
layer2.2.bn1.bias	32	0.0000
layer2.2.conv2.weight	177	0.9808
layer2.2.bn2.weight	32	0.0000

```
layer2.2.bn2.bias                    | 32             | 0.0000
layer3.0.conv1.weight                | 334            | 0.9819
layer3.0.bn1.weight                  | 64             | 0.0000
layer3.0.bn1.bias                    | 64             | 0.0000
layer3.0.conv2.weight                | 783            | 0.9788
layer3.0.bn2.weight                  | 64             | 0.0000
layer3.0.bn2.bias                    | 64             | 0.0000
layer3.0.shortcut.0.weight           | 2048           | 0.0000
layer3.0.shortcut.1.weight           | 64             | 0.0000
layer3.0.shortcut.1.bias             | 64             | 0.0000
layer3.1.conv1.weight                | 753            | 0.9796
layer3.1.bn1.weight                  | 64             | 0.0000
layer3.1.bn1.bias                    | 64             | 0.0000
layer3.1.conv2.weight                | 744            | 0.9798
layer3.1.bn2.weight                  | 64             | 0.0000
layer3.1.bn2.bias                    | 64             | 0.0000
layer3.2.conv1.weight                | 729            | 0.9802
layer3.2.bn1.weight                  | 64             | 0.0000
layer3.2.bn1.bias                    | 64             | 0.0000
layer3.2.conv2.weight                | 693            | 0.9812
layer3.2.bn2.weight                  | 64             | 0.0000
layer3.2.bn2.bias                    | 64             | 0.0000
linear.weight                        | 19             | 0.9703
linear.bias                          | 1              | 0.9000
---------------------------------------------------------------------
TOTAL NETWORK SPARSITY               | 9498           | 0.9651
```

Visualize kernels

```
# for opt in optimizers:
#     for s in sparsities:
#         visualize_kernels(results, opt=opt, s=s)
```